

SETTINGS AND PERMISSIONS

Claude Code settings

Configure Claude Code with global and project-level settings, and environment variables.

Claude Code offers a variety of settings to configure its behavior to meet your needs. You can configure Claude Code by running the `/config` command, which opens a tabbed Settings interface where you can view status information and modify configuration options. From v2.1.181, you can change a single option without opening the interface by passing `key=value` to `/config`, for example `/config verbose=true`.

Configuration scopes

Claude Code uses a scope system to determine where configurations apply and who they're shared with. Understanding scopes helps you decide how to configure Claude Code for personal use, team collaboration, or enterprise deployment.

Available scopes

Scope	Location	Who it affects	Shared with team?
Managed	Server-managed settings, plist / registry, or system-level <code>managed-settings.json</code>	All organization members for server-managed delivery; all users on the machine for plist, HKLM registry, and file delivery; the current user for HKCU registry delivery	Yes (deployed by IT)
User	<code>~/.claude/</code> directory	You, across all projects	No
Project	<code>.claude/</code> in repository	All collaborators on this repository	Yes (committed to git)
Local	<code>.claude/settings.local.json</code>	You, in this repository only	No (gitignored when Claude Code creates it)

When to use each scope

Managed scope is for:

- Security policies that must be enforced organization-wide
- Compliance requirements that can't be overridden
- Standardized configurations deployed by IT/DevOps

User scope is best for:

- Personal preferences you want everywhere (themes, editor settings)
- Tools and plugins you use across all projects
- API keys and authentication (stored securely)

Project scope is best for:

- Team-shared settings (permissions, hooks, MCP servers)
- Plugins the whole team should have
- Standardizing tooling across collaborators

Local scope is best for:

- Personal overrides for a specific project
- Testing configurations before sharing with the team
- Machine-specific settings that won't work for others

How scopes interact

When the same setting appears in multiple scopes, Claude Code applies them in priority order:

1. **Managed** (highest): can't be overridden by anything
2. **Command line arguments**: temporary session overrides
3. **Local**: overrides project and user settings
4. **Project**: overrides user settings
5. **User** (lowest): applies when nothing else specifies the setting

For example, if your user settings set `spinnerTipsEnabled` to `true` and project settings set it to `false`, the project value applies. Permission rules behave differently because they merge across

scopes rather than override. See [Settings precedence](#).

What uses scopes

Scopes apply to many Claude Code features:

Feature	User location	Project location	Local location
Settings	<code>~/.claude/settings.js</code> <code>on</code>	<code>.claude/settings.json</code>	<code>.claude/settings.local.js</code> <code>on</code>
Subagents	<code>~/.claude/agents/</code>	<code>.claude/agents/</code>	None
MCP servers	<code>~/.claude.json</code>	<code>.mcp.json</code>	<code>~/.claude.json</code> (per-project)
Plugins	<code>~/.claude/settings.js</code> <code>on</code>	<code>.claude/settings.json</code>	<code>.claude/settings.local.js</code> <code>on</code>
CLAUDE.md	<code>~/.claude/CLAUDE.md</code>	<code>CLAUDE.md</code> or <code>.claude/CLAUDE.md</code>	<code>CLAUDE.local.md</code>

On Windows, paths shown as `~/.claude` resolve to `%USERPROFILE%\ .claude` .

Settings files

The `settings.json` file is the official mechanism for configuring Claude Code through hierarchical settings:

- **User settings** are defined in `~/.claude/settings.json` and apply to all projects.
- **Project settings** are saved in your project directory:
 - `.claude/settings.json` for settings that are checked into source control and shared with your team
 - `.claude/settings.local.json` for settings that are not checked in, useful for personal preferences and experimentation. When Claude Code creates `.claude/settings.local.json` , it configures git to ignore the file. If you create the file yourself, add it to your gitignore manually.
- **Managed settings:** For organizations that need centralized control, Claude Code supports multiple delivery mechanisms for managed settings. All use the same JSON format and cannot

be overridden by user or project settings:

- **Server-managed settings:** delivered from Anthropic's servers via the Claude.ai admin console. See [server-managed settings](#).
- **MDM/OS-level policies:** delivered through native device management on macOS and Windows:
 - macOS: `com.anthropic.claudecode` managed preferences domain. The plist's top-level keys mirror `managed-settings.json`, with nested settings as dictionaries and arrays as plist arrays. Deploy via configuration profiles in Jamf, Iru (Kandji), or similar MDM tools.
 - Windows: `HKLM\SOFTWARE\Policies\ClaudeCode` registry key with a `Settings` value (REG_SZ or REG_EXPAND_SZ) containing JSON (deployed via Group Policy or Intune)
 - Windows (user-level): `HKCU\SOFTWARE\Policies\ClaudeCode` (lowest policy priority, only used when no admin-level source exists)
- **File-based:** `managed-settings.json` and `managed-mcp.json` deployed to system directories:
 - macOS: `/Library/Application Support/ClaudeCode/`
 - Linux and WSL: `/etc/claude-code/`
 - Windows: `C:\Program Files\ClaudeCode\`

 The legacy Windows path `C:\ProgramData\ClaudeCode\managed-settings.json` is no longer supported as of v2.1.75. Administrators who deployed settings to that location must migrate files to `C:\Program Files\ClaudeCode\managed-settings.json`.

File-based managed settings also support a drop-in directory at `managed-settings.d/` in the same system directory alongside `managed-settings.json`. This lets separate teams deploy independent policy fragments without coordinating edits to a single file.

Following the systemd convention, `managed-settings.json` is merged first as the base, then all `*.json` files in the drop-in directory are sorted alphabetically and merged on top. Later files override earlier ones for scalar values, arrays are concatenated and de-duplicated, and objects are deep-merged. Hidden files starting with `.` are ignored.

Use numeric prefixes to control merge order, for example `10-telemetry.json` and `20-security.json`.

See [managed settings](#) and [Managed MCP configuration](#) for details.

This [repository](#) includes starter deployment templates for Jamf, Iru (Kandji), Intune, and Group Policy. Use these as starting points and adjust them to fit your needs.

ⓘ Managed deployments can also restrict **plugin marketplace additions** using `strictKnownMarketplaces` . For more information, see [Managed marketplace restrictions](#).

- **Other configuration** is stored in `~/.claude.json` . This file contains your OAuth session, **MCP server** configurations for user and local scopes, per-project state (allowed tools, trust settings), and various caches. Project-scoped MCP servers are stored separately in `.mcp.json` .

ⓘ Claude Code automatically creates timestamped backups of configuration files and retains the five most recent backups to prevent data loss.

Example settings.json

```
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "permissions": {
    "allow": [
      "Bash(npm run lint)",
      "Bash(npm run test *)",
      "Read(~/.zshrc)"
    ],
    "deny": [
      "Bash(curl *)",
      "Read(./env)",
      "Read(./env.*)",
      "Read(./secrets/**)"
    ]
  },
  "env": {
    "CLAUDE_CODE_ENABLE_TELEMETRY": "1",
    "OTEL_METRICS_EXPORTER": "otlp"
  },
  "companyAnnouncements": [
    "Welcome to Acme Corp! Review our code guidelines at docs.acme.com",
    "Reminder: Code reviews required for all PRs",
    "New security policy in effect"
  ]
}
```

The `$schema` line in the example above points to the **[official JSON schema](#)** for Claude Code settings. Adding it to your `settings.json` enables autocomplete and inline validation in VS Code, Cursor, and any other editor that supports JSON schema validation.

The published schema is updated periodically and may not include settings added in the most recent CLI releases, so a validation warning on a recently documented field does not necessarily mean your configuration is invalid.

When edits take effect

Claude Code watches your settings files and reloads them when they change, so edits to most keys apply to the running session without a restart. This includes `permissions` , `hooks` , and credential helpers like `apiKeyHelper` . The reload covers user, project, local, and managed settings, and the `ConfigChange` hook fires for each detected change.

A few keys are read once at session start and apply on the next restart instead:

- `model` : use `/model` to switch mid-session
- `outputStyle` : part of the system prompt, which is rebuilt on `/clear` or restart

Invalid entries in managed settings

Managed settings parse tolerantly. When a managed configuration contains an entry that fails schema validation, Claude Code strips that entry, records a warning, and enforces every remaining valid policy. A single typo cannot disable the rest of your organization’s policy.

This behavior is consistent across all three delivery mechanisms: **server-managed settings**, plist and registry policies deployed through MDM, and `managed-settings.json` files. Requires Claude Code v2.1.169 or later.

Security-enforcement fields are handled per field instead of being stripped wholesale when they are present but invalid:

Field	Behavior when present but invalid
<code>allowedMcpServers</code>	Enforced as an empty allowlist, so no MCP servers are admitted until the value is fixed. An individual invalid entry is stripped and the valid subset is enforced.
<code>allowManagedMcpServersOnly</code>	Treated as <code>true</code> .
<code>availableModels</code>	Enforced as an empty allowlist, so only the Default model is available until the value is fixed. An individual non-string entry is stripped and the valid subset is enforced. Applies in v2.1.175 and later.
<code>enforceAvailableModels</code>	Treated as <code>true</code> . Applies in v2.1.175 and later.

Field	Behavior when present but invalid
forceLoginOrgUUID	No organization is permitted to log in until the value is fixed.
deniedMcpServers	An individual invalid entry is stripped and the valid subset is enforced. A wholly invalid value is dropped with a warning, since denying every server would block servers the policy never named.
sandbox.credentials	An individual invalid entry in files or envVars is stripped with a warning and the valid subset is enforced. A wholly invalid credentials value is dropped with a warning while the rest of sandbox still applies. Applies in v2.1.191 and later.

requiredMinimumVersion and requiredMaximumVersion fail open by design: an invalid value is stripped rather than enforced, so a bad policy push cannot prevent Claude Code from starting.

Validation errors surface in three places:

- Interactive sessions show a dialog at startup listing the invalid entries.
- Headless runs with -p print a summary to stderr.
- claude doctor lists each invalid entry with its source and field.

Validate policy changes by running claude doctor on a test machine before deploying them fleet-wide.

This tolerance applies only to managed settings. User, project, and local settings files remain strict: a file that fails validation is rejected as a whole and reported.

Available settings

settings.json supports a number of options:

Key	Description	Example
advisorModel	Model for the server-side <u>advisor tool</u> . Accepts a model alias such as "opus" , "sonnet" , or "fable" (v2.1.170+), or a full model ID. Written automatically when you run /advisor . Unset to disable the advisor. Requires Claude Code v2.1.98 or later	"opus"
agent	Run the main thread as a named subagent, and set the default agent for sessions dispatched from claude agents . Applies that subagent's system prompt, tool restrictions, and model. See <u>Invoke subagents explicitly</u>	"code-reviewer"
agentPushNotifEnabled	Default: false . When <u>Remote Control</u> is connected, allow Claude to send proactive push notifications to your phone,	true

Key	Description	Example
	for example when a long task finishes. Appears in <code>/config</code> as Push when Claude decides . See <u>Mobile push notifications</u> . Requires Claude Code v2.1.119 or later	
<code>allowAllClaudeAiMcp</code> <code>s</code>	(Managed settings only) Load claude.ai connectors alongside a deployed <code>managed-mcp.json</code> , which otherwise takes exclusive control and suppresses them. See <u>Managed MCP configuration</u>	<code>true</code>
<code>allowedChannelPlugi</code> <code>ns</code>	(Managed settings only) Allowlist of channel plugins that may push messages. Replaces the default Anthropic allowlist when set. Undefined = fall back to the default, empty array = block all channel plugins. Requires <code>channelsEnabled: true</code> . See <u>Restrict which channel plugins can run</u>	<code>[{ "marketplace": "claude-plugins-official", "plugin": "telegram" }]</code>
<code>allowedHttpHookUrl</code> <code>s</code>	Allowlist of URL patterns that HTTP hooks may target. Supports <code>*</code> as a wildcard. When set, hooks with non-matching URLs are blocked. Undefined = no restrictions, empty array = block all HTTP hooks. Arrays merge across settings sources. See <u>Hook configuration</u>	<code>["https://hooks.example.com/*"]</code>
<code>allowedMcpServers</code>	When set in managed-settings.json, allowlist of MCP servers users can configure. Undefined = no restrictions, empty array = lockdown. Applies to all scopes. Denylist takes precedence. See <u>Managed MCP configuration</u>	<code>[{ "serverName": "github" }]</code>
<code>allowManagedHooksOn</code> <code>ly</code>	(Managed settings only) Only managed hooks, SDK hooks, and hooks from plugins force-enabled in managed settings <code>enabledPlugins</code> are loaded. User, project, and all other plugin hooks are blocked. See <u>Hook configuration</u>	<code>true</code>
<code>allowManagedMcpServ</code> <code>ersOnly</code>	(Managed settings only) Only <code>allowedMcpServers</code> from managed settings are respected. <code>deniedMcpServers</code> still merges from all sources. Users can still add MCP servers, but only the admin-defined allowlist applies. See <u>Managed MCP configuration</u>	<code>true</code>
<code>allowManagedPermiss</code> <code>ionRulesOnly</code>	(Managed settings only) Prevent user and project settings from defining <code>allow</code> , <code>ask</code> , or <code>deny</code> permission rules. Only rules in managed settings apply. See <u>Managed-only settings</u>	<code>true</code>
<code>alwaysThinkingEnabl</code> <code>ed</code>	Enable <u>extended thinking</u> by default for all sessions. Typically configured via the <code>/config</code> command rather than editing directly. To force thinking off regardless of this setting, set <code>MAX_THINKING_TOKENS=0</code> in <code>env</code> , which disables thinking on the Anthropic API except on Fable 5, which cannot have thinking turned off. On <u>third-party providers</u> this omits the <code>thinking</code> parameter instead, and adaptive-reasoning models may still think	<code>true</code>

Key	Description	Example
<code>apiKeyHelper</code>	Custom command, run through the system shell (<code>/bin/sh</code> on macOS and Linux, <code>cmd</code> on Windows), to generate an auth value. This value will be sent as <code>X-API-Key</code> and <code>Authorization: Bearer</code> headers for model requests. Set the refresh interval with <code>CLAUDE_CODE_API_KEY_HELPER_TTL_MS</code>	<code>/bin/generate_temp_api_key.sh</code>
<code>attribution</code>	Customize attribution for git commits and pull requests. See Attribution settings	<code>{"commit": "🤖 Generated with Claude Code", "pr": ""}</code>
<code>autoCompactEnabled</code>	Default: <code>true</code> . Automatically compact the conversation when context approaches the limit. Appears in <code>/config</code> as Auto-compact . To disable via environment variable, set <code>DISABLE_AUTO_COMPACT</code> in <code>env</code>	<code>false</code>
<code>autoMemoryDirectory</code>	Custom directory for auto memory storage. Accepts an absolute path or a <code>~/</code> -prefixed path. From project or local settings, this is honored only after you accept the workspace trust dialog, since a cloned repository can supply this file	<code>"~/my-memory-dir"</code>
<code>autoMemoryEnabled</code>	Default: <code>true</code> . Enable auto memory . When <code>false</code> , Claude does not read from or write to the auto memory directory. You can also toggle this with <code>/memory</code> during a session. To disable via environment variable, set <code>CLAUDE_CODE_DISABLE_AUTO_MEMORY</code> in <code>env</code>	<code>false</code>
<code>autoMode</code>	Customize what the auto mode classifier blocks and allows. Contains <code>environment</code> , <code>allow</code> , <code>soft_deny</code> , and <code>hard_deny</code> arrays of prose rules. Include the literal string <code>"\$defaults"</code> in an array to inherit the built-in rules at that position. See Configure auto mode . Not read from shared project settings	<code>{"soft_deny": ["\$defaults", "Never run terraform apply"]}</code>
<code>autoScrollEnabled</code>	Default: <code>true</code> . In fullscreen rendering , follow new output to the bottom of the conversation. Appears in <code>/config</code> as Auto-scroll . Permission prompts still scroll to view when this is off	<code>false</code>
<code>autoUpdatesChannel</code>	Default: <code>"latest"</code> . Release channel to follow for updates. Use <code>"stable"</code> for a version that is typically about one week old and skips versions with major regressions, or <code>"latest"</code> for the most recent release. To disable auto-updates entirely, set <code>DISABLE_AUTOUPDATER</code> in <code>env</code>	<code>"stable"</code>
<code>availableModels</code>	Restrict which models users can select for the main session, subagents , skills , and the advisor . Does not affect the Default option unless <code>enforceAvailableModels</code> is also set. See Restrict model selection	<code>["sonnet", "haiku"]</code>
<code>awaySummaryEnabled</code>	Show a one-line session recap when you return to the terminal after a few minutes away. Set to <code>false</code> or turn off	<code>true</code>

Key	Description	Example
	Session recap in <code>/config</code> to disable. Same as <code>CLAUDE_CODE_ENABLE_AWAY_SUMMARY</code>	
<code>awsAuthRefresh</code>	Custom script that modifies the <code>.aws</code> directory (see advanced credential configuration)	<code>aws sso login --profile myprofile</code>
<code>awsCredentialExport</code>	Custom script that outputs JSON with AWS credentials (see advanced credential configuration)	<code>/bin/generate_aws_grant.sh</code>
<code>axScreenReader</code>	Render screen-reader friendly output: flat text without decorative borders or animations. Screen-reader mode always uses the classic renderer, so the <code>tui</code> setting has no effect while it is active. The <code>CLAUDE_AX_SCREEN_READER</code> environment variable and the <code>--ax-screen-reader</code> flag take precedence. Requires Claude Code v2.1.181 or later	<code>true</code>
<code>blockedMarketplaces</code>	(Managed settings only) Blocklist of marketplace sources. Enforced on marketplace add and on plugin install, update, refresh, and auto-update, so a marketplace added before the policy was set cannot be used to fetch plugins. Blocked sources are checked before downloading, so they never touch the filesystem. See Managed marketplace restrictions	<code>[{ "source": "github", "repo": "untrusted/plugins" }]</code>
<code>channelsEnabled</code>	(Managed settings only) Allow channels for the organization. On claude.ai Team and Enterprise plans, channels are blocked when this is unset or <code>false</code> . For Anthropic Console accounts using API key authentication, channels are allowed by default unless your organization deploys managed settings, in which case this key must be set to <code>true</code>	<code>true</code>
<code>claudeMd</code>	(Managed settings only) CLAUDE.md-style instructions injected as organization-managed memory. Only honored when set in managed or policy settings and ignored in user, project, and local settings. See organization-wide CLAUDE.md	<code>"Always run make lint before committing."</code>
<code>claudeMdExcludes</code>	Glob patterns or absolute paths of <code>CLAUDE.md</code> files to skip when loading memory . Patterns match against absolute file paths. Only applies to user, project, and local memory; managed policy files cannot be excluded	<code>["**/vendor/**/CLAUDE.md"]</code>
<code>cleanupPeriodDays</code>	Default: <code>30</code> days, minimum <code>1</code> . Session files older than this period are deleted at startup. Setting to <code>0</code> is rejected with a validation error. Also controls the age cutoff for automatic removal of orphaned subagent worktrees at startup. To disable transcript writes entirely, set the <code>CLAUDE_CODE_SKIP_PROMPT_HISTORY</code> environment variable, or in non-interactive mode (<code>-p</code>) use the <code>--no-session-persistence</code> flag or the <code>persistSession: false</code> SDK option.	<code>20</code>
<code>companyAnnouncement</code>	Announcement to display to users at startup. If multiple	<code>["Welcome to Acme Corp!</code>

Key	Description	Example
s	announcements are provided, they will be cycled through at random.	Review our code guidelines at docs.acme.com"]
defaultShell	Default: "bash" , or "powershell" on Windows when Bash isn't available. Default shell for input-box ! commands. Accepts "bash" or "powershell" . Setting "powershell" routes interactive ! commands through PowerShell on Windows. Requires CLAUDE_CODE_USE_POWERSHELL_TOOL=1 . See PowerShell tool	"powershell"
deniedMcpServers	When set in managed-settings.json, denylist of MCP servers that are explicitly blocked. Applies to all scopes including managed servers. Denylist takes precedence over allowlist. See Managed MCP configuration	[{ "serverName": "filesystem" }]
disableAgentView	Set to true to turn off background agents and agent view: claude agents , --bg , /background , and the on-demand supervisor. Typically set in managed settings . Equivalent to setting CLAUDE_CODE_DISABLE_AGENT_VIEW to 1	true
disableAllHooks	Disable all hooks and any custom status line	true
disableArtifact	Set to true to disable the Artifact tool, which publishes session output as a private web page on claude.ai. Equivalent to setting CLAUDE_CODE_DISABLE_ARTIFACT to 1	true
disableAutoMode	Set to "disable" to prevent auto mode from being activated. Removes auto from the Shift+Tab cycle and rejects --permission-mode auto at startup. Most useful in managed settings where users cannot override it	"disable"
disableBundledSkills	Set to true to disable the skills and workflows that ship with Claude Code: bundled skills and workflows are removed entirely, while built-in slash commands like /init stay typable but are hidden from the model. Skills from plugins, .claude/skills/ , and .claude/commands/ are unaffected. Equivalent to setting CLAUDE_CODE_DISABLE_BUNDLED_SKILLS to 1	true
disableClaudeAiConnectors	Disable claude.ai MCP connectors so they are not auto-fetched or connected. Set in any settings scope. true in any source takes precedence, so a checked-in project .claude/settings.json can opt a repo out of cloud connectors, but a project-level false cannot override a user- or policy-level true . Servers passed explicitly via --mcp-config are unaffected. To deny individual connectors instead of all of them, use deniedMcpServers . Requires Claude Code v2.1.182 or later	true

Key	Description	Example
<code>disableDeepLinkRegistration</code>	Set to <code>"disable"</code> to prevent Claude Code from registering the <code>claude-cli://</code> protocol handler with the operating system on startup. Deep links let external tools open a Claude Code session with a pre-filled prompt. Useful in environments where protocol handler registration is restricted or managed separately	<code>"disable"</code>
<code>disabledMcpjsonServers</code>	List of specific MCP servers from <code>.mcp.json</code> files to reject	<code>["filesystem"]</code>
<code>disableRemoteControl</code>	Disable Remote Control : blocks <code>claude remote-control</code> , the <code>--remote-control</code> flag, auto-start, and the in-session toggle. Typically placed in managed settings for per-device MDM enforcement, but works from any scope. Requires Claude Code v2.1.128 or later	<code>true</code>
<code>disableSkillShellExecution</code>	Disable inline shell execution for <code>!`...`</code> and <code>``!</code> blocks in skills and custom commands from user, project, plugin, or additional-directory sources. Commands are replaced with <code>[shell command execution disabled by policy]</code> instead of being run. Bundled and managed skills are not affected. Most useful in managed settings where users cannot override it	<code>true</code>
<code>disableWorkflows</code>	Default: <code>false</code> . Disable dynamic workflows and the bundled workflow commands. Equivalent to setting <code>CLAUDE_CODE_DISABLE_WORKFLOWS</code> to <code>1</code>	<code>true</code>
<code>editorMode</code>	Default: <code>"normal"</code> . Key binding mode for the input prompt: <code>"normal"</code> or <code>"vim"</code> . Appears in <code>/config</code> as Editor mode	<code>"vim"</code>
<code>effortLevel</code>	Persist the effort level across sessions. Accepts <code>"low"</code> , <code>"medium"</code> , <code>"high"</code> , or <code>"xhigh"</code> . Written automatically when you run <code>/effort</code> with one of those values. <code>--effort</code> and <code>CLAUDE_CODE_EFFORT_LEVEL</code> override this for one session. See Adjust effort level for supported models	<code>"xhigh"</code>
<code>enableAllProjectMcpServers</code>	Automatically approve all MCP servers defined in project <code>.mcp.json</code> files	<code>true</code>
<code>enabledMcpjsonServers</code>	List of specific MCP servers from <code>.mcp.json</code> files to approve	<code>["memory", "github"]</code>
<code>enforceAvailableModels</code>	Extend the <code>availableModels</code> allowlist to the Default model. When <code>true</code> in managed settings and <code>availableModels</code> is a non-empty array, the Default option falls back to the first allowlisted entry that is available. Has no effect when <code>availableModels</code> is unset or empty. See Enforce the allowlist for the Default model . Requires Claude Code v2.1.175 or later	<code>true</code>
<code>env</code>	Environment variables applied to every session and to subprocesses Claude Code spawns from it. As of v2.1.143,	<code>{"F00": "bar"}</code>

Key	Description	Example
	<code>NO_COLOR</code> and <code>FORCE_COLOR</code> set here are passed to subprocesses but do not change Claude Code's own interface colors. Set those in your shell before launching <code>claude</code> to change interface colors	
<code>fallbackModel</code>	Fallback model(s) to try in order when the primary model is overloaded or unavailable. Claude Code switches to the next available model in the chain for the rest of the turn and shows a notice. <code>"default"</code> expands to the default model. Chains are capped at three models; extra entries are ignored. Unlike most array settings, this key does not merge across settings files: the highest-precedence file that defines it supplies the entire chain. The <code>--fallback-model</code> flag overrides this for one session. See Fallback model chains	<code>["claude-sonnet-4-6", "claude-haiku-4-5"]</code>
<code>fastModePerSessionOptIn</code>	When <code>true</code> , fast mode does not persist across sessions. Each session starts with fast mode off, requiring users to enable it with <code>/fast</code> . The user's fast mode preference is still saved. See Require per-session opt-in	<code>true</code>
<code>feedbackSurveyRate</code>	Probability (0-1) that the session quality survey appears when eligible. Set to <code>0</code> to suppress entirely, or set <code>CLAUDE_CODE_DISABLE_FEEDBACK_SURVEY</code> in <code>env</code> . Useful when using Bedrock, Vertex, or Foundry where the default sample rate does not apply	<code>0.05</code>
<code>fileCheckpointingEnabled</code>	Default: <code>true</code> . Snapshot files before each edit so <code>/rewind</code> can restore them. Appears in <code>/config</code> as Rewind code (checkpoints) . To disable via environment variable, set <code>CLAUDE_CODE_DISABLE_FILE_CHECKPOINTING</code> in <code>env</code>	<code>false</code>
<code>fileSuggestion</code>	Configure a custom script for <code>@</code> file autocomplete. See File suggestion settings	<code>{"type": "command", "command": "~/.claude/file-suggestion.sh"}</code>
<code>footerLinksRegexes</code>	Render extra clickable badges in the footer when a regex matches turn output. Each entry has a <code>pattern</code> , a <code>url</code> template with <code>{name}</code> placeholders filled from named capture groups, and an optional <code>label</code> . Read from user, <code>-settings</code> flag, and managed settings only. See Footer link badges for URL constraints, scheme allowlist, and limits. Requires Claude Code v2.1.176 or later	<code>[{"type": "regex", "pattern": "\\b(?<key>PROJ-\\d+)\\b", "url": "https://issues.example.com/browse/{key}", "label": "{key}"}]</code>
<code>forceLoginMethod</code>	Use <code>claudeai</code> to restrict login to Claude.ai accounts, <code>console</code> to restrict login to Claude Console accounts. When set in managed settings, sessions authenticated by <code>ANTHROPIC_API_KEY</code> , <code>ANTHROPIC_AUTH_TOKEN</code> , or <code>apiKeyHelper</code> are blocked at startup, since neither value can be satisfied without first-party OAuth. Third-party provider sessions such as Bedrock, Vertex, and Foundry are	<code>claudeai</code>

Key	Description	Example
	v2.1.176, when not set, session titles match the language of your conversation	
maxSkillDescriptionChars	Default: 1536 . Per-skill character cap on the combined description and when_to_use text in the skill listing Claude sees each turn. Text longer than this is truncated. Raise to keep long descriptions intact at the cost of more context per turn; lower to fit more skills under skillListingBudgetFraction . Requires Claude Code v2.1.105 or later	2048
minimumVersion	Floor that prevents background auto-updates and claude update from installing a version below this one. Switching from the "latest" channel to "stable" via /config prompts you to stay on the current version or allow the downgrade. Choosing to stay sets this value. Also useful in managed settings to pin an organization-wide minimum. For a hard floor that blocks startup entirely, see requiredMinimumVersion	"2.1.100"
model	Override the default model to use for Claude Code. -- model and ANTHROPIC_MODEL override this for one session	"claude-sonnet-4-6"
modelOverrides	Map Anthropic model IDs to provider-specific model IDs such as Bedrock inference profile ARNs. Each model picker entry uses its mapped value when calling the provider API. See Override model IDs per version	{"claude-opus-4-6": "arn:aws:bedrock:..."}
otelHeadersHelper	Script to generate dynamic OpenTelemetry headers. Runs at startup and periodically. Set the refresh interval with CLAUDE_CODE_OTEL_HEADERS_HELPER_DEBOUNCE_MS . See Dynamic headers	/bin/generate_otel_headers.sh
outputStyle	Configure an output style to adjust the system prompt. See output styles documentation	"Explanatory"
parentSettingsBehavior	(Managed settings only) Default: "first-wins" . Controls whether managed settings supplied programmatically by an embedding host process, such as the Agent SDK or an IDE extension, apply when an admin-deployed managed tier is also present. "first-wins" : the parent-supplied settings are dropped and only the admin tier applies. "merge" : the parent-supplied settings apply under the admin tier, filtered so they can tighten policy but not loosen it. Has no effect when no admin tier is deployed. Requires Claude Code v2.1.133 or later	"merge"
permissions	See table below for structure of permissions.	
plansDirectory	Default: ~/.claude/plans . Customize where plan files are stored. Path is relative to project root.	"./plans"
pluginSuggestionMar	(Managed settings only) Marketplace names whose plugins	["acme-corp-plugins"]

Key	Description	Example
ketplaces	can appear as contextual install suggestions. No marketplace-declared suggestions surface without this allowlist; the built-in first-party frontend-design tip is unaffected. Suggestions come from each plugin's <code>relevance</code> declaration in its marketplace entry. A name only takes effect when the marketplace is registered on the machine and its registered source is also declared in managed settings, either as the <code>extraKnownMarketplaces</code> entry for that name or as an entry of <code>strictKnownMarketplaces</code> . A marketplace registered from a different source under an allowlisted name is ignored. The official marketplace is exempt from the source requirement: allowlisting its name alone suffices, since that name can only register from the official Anthropic source.	
pluginTrustMessage	(Managed settings only) Custom message appended to the plugin trust warning shown before installation. Use this to add organization-specific context, for example to confirm that plugins from your internal marketplace are vetted.	"All plugins from our marketplace are approved by IT"
policyHelper	Admin-deployed executable that computes managed settings dynamically at startup. Only honored from MDM or a system <code>managed-settings.json</code> file. See <u>Compute managed settings with a policy helper</u> . Requires Claude Code v2.1.136 or later	<code>{"path": "/usr/local/bin/claude-policy"}</code>
preferredNotifChannel	Default: <code>"auto"</code> . Method for task-complete and permission-prompt notifications: <code>"auto"</code> , <code>"terminal_bell"</code> , <code>"iterm2"</code> , <code>"iterm2_with_bell"</code> , <code>"kitty"</code> , <code>"ghostty"</code> , or <code>"notifications_disabled"</code> . <code>"auto"</code> sends a desktop notification in iTerm2, Ghostty, and Kitty and does nothing in other terminals. Set <code>"terminal_bell"</code> to ring the bell character in any terminal. Appears in <code>/config</code> as Notifications . See <u>Get a terminal bell or notification</u>	<code>"terminal_bell"</code>
prefersReducedMotion	Reduce or disable UI animations (spinners, shimmer, flash effects) for accessibility	<code>true</code>
prUrlTemplate	URL template for the PR badge shown in the footer and in tool-result summaries. Substitutes <code>{host}</code> , <code>{owner}</code> , <code>{repo}</code> , <code>{number}</code> , and <code>{url}</code> from the <code>gh</code> -reported PR URL. Use to point PR links at an internal code-review tool instead of <code>github.com</code> . Does not affect <code>#123</code> autolinks in Claude's prose	<code>"https://reviews.example.com/{owner}/{repo}/pull/{number}"</code>
remoteControlAtStartup	Connect <u>Remote Control</u> automatically when each interactive session starts, instead of waiting for <code>/remote-control</code> . Set to <code>true</code> to always auto-connect, <code>false</code> to never auto-connect, or leave unset to follow your organization's default. Appears in <code>/config</code> as Enable Remote Control for all sessions . See <u>Enable Remote Control for all sessions</u>	<code>false</code>

Key	Description	Example
<code>requiredMaximumVersion</code>	Managed settings only. Maximum Claude Code version allowed to start. If the running version is newer, Claude Code exits at startup and instructs the user to install an approved version through the organization's approved method; <code>claude install <version></code> may also work. Background auto-updates and <code>claude update</code> skip versions above the ceiling, so an in-range installation stays in range. <code>claude update</code> , <code>claude install</code> , and <code>claude doctor</code> keep working above the ceiling so users can recover. Versions that predate this setting ignore it	"2.1.150"
<code>requiredMinimumVersion</code>	Managed settings only. Minimum Claude Code version required to start. If the running version is older, Claude Code exits at startup and instructs the user to update through the organization's approved method. <code>claude update</code> , <code>claude install</code> , and <code>claude doctor</code> keep working below the floor so users can recover. Differs from <code>minimumVersion</code> , which prevents downgrades but never blocks startup. Versions that predate this setting ignore it	"2.1.150"
<code>respectGitignore</code>	Default: <code>true</code> . Control whether the <code>@</code> file picker respects <code>.gitignore</code> patterns. When <code>true</code> , files matching <code>.gitignore</code> patterns are excluded from suggestions	<code>false</code>
<code>respondToBashCommands</code>	Default: <code>true</code> . Whether Claude responds after an input-box <code>!</code> shell command runs. Set to <code>false</code> to add the command output to context without a response. See Shell mode with ! prefix . Requires Claude Code v2.1.186 or later	<code>false</code>
<code>showClearContextOnPlanAccept</code>	Default: <code>false</code> . Show the "clear context" option on the plan accept screen. Set to <code>true</code> to restore the option	<code>true</code>
<code>showThinkingSummaries</code>	Default: <code>false</code> . Show <u>extended thinking</u> summaries in interactive sessions. When unset or <code>false</code> , thinking blocks are redacted by the API and shown as a collapsed stub. Redaction only changes what you see, not what the model generates: to reduce thinking spend, <u>lower the budget or disable thinking</u> instead. This setting has no effect in non-interactive mode (<code>-p</code>), the Agent SDK, or IDE extensions such as VS Code	<code>true</code>
<code>showTurnDuration</code>	Default: <code>true</code> . Show turn duration messages after responses, e.g. "Cooked for 1m 6s". Appears in <code>/config</code> as Show turn duration	<code>false</code>
<code>skillListingBudgetFraction</code>	Default: <code>0.01</code> (1%). Fraction of the model's context window reserved for the <u>skill listing</u> Claude sees each turn. When the listing exceeds the budget, descriptions for the least-used skills are collapsed to bare names so Claude can still invoke them but won't see why. Raise to keep more descriptions visible at the cost of more context per turn.	<code>0.02</code>

Key	Description	Example
	<code>/doctor</code> shows the current truncation count and which skills are affected. Requires Claude Code v2.1.105 or later	
<code>skillOverrides</code>	Per-skill visibility overrides keyed by skill name. Value is <code>"on"</code> , <code>"name-only"</code> , <code>"user-invocable-only"</code> , or <code>"off"</code> . Lets you hide or collapse a skill without editing its SKILL.md. Does not apply to plugin skills, which are managed through <code>/plugin</code> . The <code>/skills</code> menu writes these to <code>.claude/settings.local.json</code> . See Override skill visibility from settings . Requires Claude Code v2.1.129 or later	<pre>{"legacy-context": "name-only", "deploy": "off"}</pre>
<code>skipWebFetchPreflight</code>	Skip the WebFetch domain safety check that sends each requested hostname to <code>api.anthropic.com</code> before fetching. Set to <code>true</code> in environments that block traffic to Anthropic, such as Bedrock, Vertex AI, or Foundry deployments with restrictive egress. When skipped, WebFetch attempts any URL without consulting the blocklist	<code>true</code>
<code>spinnerTipsEnabled</code>	Default: <code>true</code> . Show tips in the spinner while Claude is working. Set to <code>false</code> to disable tips	<code>false</code>
<code>spinnerTipsOverride</code>	Override spinner tips with custom strings. <code>tips</code> : array of tip strings. <code>excludeDefault</code> : if <code>true</code> , only show custom tips; if <code>false</code> or absent, custom tips are merged with built-in tips	<pre>{ "excludeDefault": true, "tips": ["Use our internal tool X"] }</pre>
<code>spinnerVerbs</code>	Customize the action verbs shown while a turn is in progress. Set <code>mode</code> to <code>"replace"</code> to use only your verbs, or <code>"append"</code> to add them to the defaults	<pre>{"mode": "append", "verbs": ["Pondering", "Crafting"]}</pre>
<code>sshConfigs</code>	SSH connections to show in the Desktop environment dropdown. Each entry requires <code>id</code> , <code>name</code> , and <code>sshHost</code> ; <code>sshPort</code> , <code>sshIdentityFile</code> , and <code>startDirectory</code> are optional. When set in managed settings, connections are read-only for users. Read from managed and user settings only	<pre>[{"id": "dev-vm", "name": "Dev VM", "sshHost": "user@dev.example.com"}]</pre>
<code>statusLine</code>	Configure a custom status line to display context. See statusLine documentation	<pre>{"type": "command", "command": "~/.claude/statusline.s h"}</pre>
<code>strictKnownMarketplaces</code>	(Managed settings only) Allowlist of plugin marketplace sources. Undefined = no restrictions, empty array = lockdown. Enforced on marketplace add and on plugin install, update, refresh, and auto-update, so a marketplace added before the policy was set cannot be used to fetch plugins. See Managed marketplace restrictions	<pre>[{ "source": "github", "repo": "acme- corp/plugins" }]</pre>
<code>strictPluginOnlyCustomization</code>	(Managed settings only) Block skills, agents, hooks, and MCP servers from user and project sources, so they can only come from plugins or managed settings. <code>true</code> locks	<pre>["skills", "hooks"]</pre>

Key	Description	Example
	all four surfaces; an array locks only the named ones. See strictPluginOnlyCustomization	
<code>syntaxHighlightingDisabled</code>	Disable syntax highlighting in diffs, code blocks, and file previews	<code>true</code>
<code>teammateMode</code>	Default: <code>in-process</code> . How agent team teammates display: <code>in-process</code> , <code>auto</code> (split panes when running inside tmux or iTerm2, in-process otherwise), <code>tmux</code> (split panes using tmux or iTerm2, detected from your terminal), or <code>iterm2</code> (iTerm2 native split panes via the <code>it2</code> CLI, added in v2.1.186). The default changed from <code>auto</code> in v2.1.179. <code>--teammate-mode</code> overrides this for one session. See choose a display mode	<code>"auto"</code>
<code>terminalProgressBarEnabled</code>	Default: <code>true</code> . Show the terminal progress bar in supported terminals: ConEmu, Ghostty 1.2.0+, and iTerm2 3.6.6+. Appears in <code>/config</code> as Terminal progress bar	<code>false</code>
<code>theme</code>	Default: <code>"dark"</code> . Color theme for the interface: <code>"auto"</code> , <code>"dark"</code> , <code>"light"</code> , <code>"dark-daltonized"</code> , <code>"light-daltonized"</code> , <code>"dark-ansi"</code> , <code>"light-ansi"</code> , or a custom theme reference such as <code>"custom:<slug>"</code> or <code>"custom:<plugin-name>:<slug>"</code> . See Create a custom theme . Appears in <code>/config</code> as Theme	<code>"dark"</code>
<code>tui</code>	Terminal UI renderer. Use <code>"fullscreen"</code> for the flicker-free alt-screen renderer with virtualized scrollback. Use <code>"default"</code> for the classic main-screen renderer. Set via <code>/tui</code> . You can also set the CLAUDE_CODE_NO_FLICKER environment variable. Background sessions opened from agent view always use the fullscreen renderer regardless of this setting	<code>"fullscreen"</code>
<code>ultracode</code>	Turn on ultracode for the session. Session-only and not read from <code>settings.json</code> . Set through <code>/effort ultracode</code> , <code>-settings</code> , or an Agent SDK control request	<code>true</code>
<code>useAutoModeDuringPlan</code>	Default: <code>true</code> . Whether plan mode uses auto mode semantics when auto mode is available. Not read from shared project settings. Appears in <code>/config</code> as “Use auto mode during plan”	<code>false</code>
<code>verbose</code>	Default: <code>false</code> . Show full tool output instead of truncated summaries. Appears in <code>/config</code> as Verbose output . The <code>--verbose</code> flag overrides this for one session	<code>true</code>
<code>viewMode</code>	Default transcript view mode on startup: <code>"default"</code> , <code>"verbose"</code> , or <code>"focus"</code> . Overrides the sticky <code>/focus</code> selection when set. The <code>--verbose</code> flag overrides this for one session	<code>"verbose"</code>
<code>voice</code>	Voice dictation settings: <code>enabled</code> turns dictation on, <code>mode</code> selects <code>"hold"</code> or <code>"tap"</code> , and <code>autoSubmit</code>	<code>{ "enabled": true, "mode": "tap" }</code>

Key	Description	Example
	sends the prompt on key release in hold mode. Written automatically when you run <code>/voice</code> . Requires a Claude.ai account	
<code>voiceEnabled</code>	Legacy alias for <code>voice.enabled</code> . Prefer the <code>voice</code> object	<code>true</code>
<code>wheelScrollAccelerationEnabled</code>	Default: <code>true</code> . In fullscreen rendering , accelerate mouse-wheel scroll speed during fast scrolls. Set to <code>false</code> for a constant scroll rate per wheel notch. Requires Claude Code v2.1.174 or later	<code>false</code>
<code>workflowKeywordTriggerEnabled</code>	Default: <code>true</code> . Whether the keyword <code>ultracode</code> in a prompt triggers a dynamic workflow . Set to <code>false</code> to type the word without triggering one. The <code>ultracode</code> effort setting, <code>/workflows</code> , and saved workflow commands are unaffected. Appears in <code>/config</code> as Ultracode keyword trigger . Added in v2.1.157; before v2.1.160 the trigger keyword was <code>workflow</code>	<code>false</code>
<code>wslInheritsWindowsSettings</code>	(Windows managed settings only) When <code>true</code> , Claude Code on WSL reads managed settings from the Windows policy chain in addition to <code>/etc/claude-code</code> , with Windows sources taking priority. Only honored when set in the HKLM registry key or <code>C:\Program Files\ClaudeCode\managed-settings.json</code> , both of which require Windows admin to write. For HKCU policy to also apply on WSL, the flag must additionally be set in HKCU itself. Has no effect on native Windows	<code>true</code>

Global config settings

These settings are stored in `~/.claude.json` rather than `settings.json` . Adding them to `settings.json` will trigger a schema validation error.

 Versions before v2.1.119 also store a number of `/config` preference keys here instead of in `settings.json` , including `theme` , `verbose` , `editorMode` , `autoCompactEnabled` , and `preferredNotifChannel` .

Key	Description	Example
<code>autoConnectIde</code>	Default: <code>false</code> . Automatically connect to a running IDE when Claude Code starts from an external terminal. Appears in <code>/config</code> as Auto-connect to IDE (external terminal) when running outside a VS Code or JetBrains terminal. The <code>CLAUDE_CODE_AUTO_CONNECT_IDE</code> environment variable overrides this when set	<code>true</code>

Key	Description	Example
<code>autoInstallIdeExtension</code>	Default: <code>true</code> . Automatically install the Claude Code IDE extension when running from a VS Code terminal. Appears in <code>/config</code> as Auto-install IDE extension when running inside a VS Code or JetBrains terminal. You can also set the <code>CLAUDE_CODE_IDE_SKIP_AUTO_INSTALL</code> environment variable	<code>false</code>
<code>externalEditorContext</code>	Default: <code>false</code> . Prepend Claude's previous response as <code># -</code> commented context when you open the external editor with <code>Ctrl+G</code> . Appears in <code>/config</code> as Show last response in external editor	<code>true</code>
<code>teammateDefaultModel</code>	Default model for agent team teammates when the spawn prompt doesn't specify one. Set to a model alias such as <code>"sonnet"</code> , or <code>null</code> to inherit the lead's current <code>/model</code> selection. Appears in <code>/config</code> as Default teammate model	<code>"sonnet"</code>

Worktree settings

Configure how `--worktree` creates and manages git worktrees.

Key	Description	Example
<code>worktree.baseRef</code>	Which ref new worktrees branch from. <code>"fresh"</code> (default) branches from <code>origin/<default-branch></code> for a clean tree matching the remote. <code>"head"</code> branches from your current local <code>HEAD</code> , so unpushed commits and feature-branch state are present in the worktree. Applies to <code>--worktree</code> , the <code>EnterWorktree</code> tool, and subagent isolation	<code>"head"</code>
<code>worktree.symLinkIdirectories</code>	Directories to symlink from the main repository into each worktree to avoid duplicating large directories on disk. No directories are symlinked by default	<code>["node_modules", ".cache"]</code>
<code>worktree.sparsePaths</code>	Directories to check out in each worktree via git sparse-checkout. Only the listed directories plus root-level files are written to disk, which is faster in large monorepos	<code>["packages/my-app", "shared/utils"]</code>
<code>worktree.bgIsolation</code>	Isolation mode for background sessions . <code>"worktree"</code> (default) blocks <code>Edit / Write</code> in the main checkout until <code>EnterWorktree</code> is called. <code>"none"</code> lets background jobs edit the working copy directly. Requires Claude Code v2.1.143 or later	<code>"none"</code>

To copy gitignored files like `.env` into new worktrees, use a `.worktreeinclude` file in your project root instead of a setting.

Permission settings

Keys	Description	Example
<code>allow</code>	Array of permission rules to allow tool use. Tool-name globs are supported only in the tool position after a literal <code>mcp__<server>__</code> prefix, such as <code>mcp__github__get_*</code> ; the server segment must be glob-free. See Permission rule syntax below for pattern matching details	<code>["Bash(git diff *)"]</code>
<code>ask</code>	Array of permission rules to ask for confirmation upon tool use. See Permission rule syntax below	<code>["Bash(git push *)"]</code>
<code>deny</code>	Array of permission rules to deny tool use. Use this to exclude sensitive files from Claude Code access. Tool names accept glob patterns: <code>"*"</code> denies every tool and <code>"mcp__*"</code> denies all MCP tools. See Permission rule syntax and Bash permission limitations	<code>["WebFetch", "Bash(curl *)", "Read(./env)", "Read(./secrets/**)"]</code>
<code>additionalDirectories</code>	Additional working directories for file access. Most <code>.claude/</code> configuration is not discovered from these directories	<code>["../docs/"]</code>
<code>defaultMode</code>	Default permission mode when opening Claude Code. Valid values: <code>default</code> , <code>acceptEdits</code> , <code>plan</code> , <code>auto</code> , <code>dontAsk</code> , <code>bypassPermissions</code> . As of Claude Code v2.1.142, <code>auto</code> is ignored when set in project or local settings (<code>.claude/settings.json</code> , <code>.claude/settings.local.json</code>) so a repository cannot grant itself auto mode. Set it in <code>~/ .claude/settings.json</code> instead. The <code>--permission-mode</code> CLI flag overrides this setting for a single session	<code>"acceptEdits"</code>
<code>disableBypassPermissionsMode</code>	Set to <code>"disable"</code> to prevent <code>bypassPermissions</code> mode from being activated. This disables the <code>--dangerously-skip-permissions</code> command-line flag. Typically placed in managed settings to enforce organizational policy, but works from any scope	<code>"disable"</code>
<code>skipDangerousModePermissionPrompt</code>	Skip the confirmation prompt shown before entering bypass permissions mode via <code>--dangerously-skip-permissions</code> or <code>defaultMode: "bypassPermissions"</code> . Ignored when set in project settings (<code>.claude/settings.json</code>) to prevent untrusted repositories from auto-bypassing the prompt	<code>true</code>

Permission rule syntax

Permission rules follow the format `Tool` or `Tool(specifier)` . Rules are evaluated in order: deny rules first, then ask, then allow. The first match determines the outcome regardless of rule

specificity. See the [permission rule evaluation order](#) for details.

Quick examples:

Rule	Effect
Bash	Matches all Bash commands
Bash(npm run *)	Matches commands starting with npm run
Read(./ .env)	Matches reading the .env file
WebFetch(domain:example.com)	Matches fetch requests to example.com

For the complete rule syntax reference, including wildcard behavior, tool-specific patterns for Read, Edit, WebFetch, MCP, and Agent rules, and security limitations of Bash patterns, see [Permission rule syntax](#).

Sandbox settings

Configure advanced sandboxing behavior. Sandboxing isolates bash commands from your filesystem and network. See [Sandboxing](#) for details.

Keys	Description	Example
enabled	Enable bash sandboxing (macOS, Linux, and WSL2). Default: false	true
failIfUnavailable	Exit with an error at startup if sandbox.enabled is true but the sandbox cannot start (missing dependencies or unsupported platform). When false (default), a warning is shown and commands run unsandboxed. Intended for managed settings deployments that require sandboxing as a hard gate	true
autoAllowBashIfSandboxed	Auto-approve bash commands when sandboxed. Default: true	true
excludedCommands	Commands that should run outside of the sandbox	["docker *"]
allowUnsandboxedCommands	Allow commands to run outside the sandbox via the dangerouslyDisableSandbox parameter. When set to false, the dangerouslyDisableSandbox escape hatch is completely disabled and all commands must run sandboxed (or be in excludedCommands). Useful for enterprise policies that require strict sandboxing. Default: true	false
filesystem.allowWrite	Additional paths where sandboxed commands can write.	["/tmp/build",

Keys	Description	Example
<code>te</code>	Arrays are merged across all settings scopes: user, project, and managed paths are combined, not replaced. Also merged with paths from <code>Edit(...)</code> allow permission rules. See path prefixes below.	<code>"~/ .kube"]</code>
<code>filesystem.denyWrite</code>	Paths where sandboxed commands cannot write. Arrays are merged across all settings scopes. Also merged with paths from <code>Edit(...)</code> deny permission rules.	<code>["/etc", "/usr/local/bin"]</code>
<code>filesystem.denyRead</code>	Paths where sandboxed commands cannot read. Arrays are merged across all settings scopes. Also merged with paths from <code>Read(...)</code> deny permission rules.	<code>["~/ .aws/credentials"]</code>
<code>filesystem.allowRead</code>	Paths to re-allow reading within <code>denyRead</code> regions. Takes precedence over <code>denyRead</code> . Arrays are merged across all settings scopes. Use this to create workspace-only read access patterns.	<code>["."]</code>
<code>filesystem.allowManagedReadPathsOnly</code>	(Managed settings only) Only <code>filesystem.allowRead</code> paths from managed settings are respected. <code>denyRead</code> still merges from all sources. Default: false	<code>true</code>
<code>credentials.files</code>	Credential files or directories that sandboxed commands cannot read. Applies the same read block as <code>filesystem.denyRead</code> ; the separate key keeps credential paths grouped with <code>credentials.envVars</code> and apart from general filesystem rules. Each entry is <code>{ "path": "...", "mode": "deny" }</code> . Paths use the same prefixes as <code>filesystem.*</code> settings. Arrays are merged across all settings scopes. Only <code>deny</code> is supported. Requires Claude Code v2.1.187 or later.	<code>[{ "path": "~/ .aws/credentials", "mode": "deny" }]</code>
<code>credentials.envVars</code>	Environment variables to unset before running sandboxed commands. Each entry is <code>{ "name": "...", "mode": "deny" }</code> . Arrays are merged across all settings scopes. Only <code>deny</code> is supported. Requires Claude Code v2.1.187 or later.	<code>[{ "name": "GITHUB_TOKEN", "mode": "deny" }]</code>
<code>network.allowUnixSockets</code>	(macOS only) Unix socket paths accessible in sandbox. Ignored on Linux and WSL2, where the seccomp filter cannot inspect socket paths; use <code>allowAllUnixSockets</code> instead.	<code>["~/ .ssh/agent-socket"]</code>
<code>network.allowAllUnixSockets</code>	Allow all Unix socket connections in sandbox. On Linux and WSL2 this is the only way to permit Unix sockets, since it skips the seccomp filter that otherwise blocks <code>socket(AF_UNIX, ...)</code> calls. Default: false	<code>true</code>
<code>network.allowLocalBinding</code>	Allow binding to localhost ports (macOS only). Default: false	<code>true</code>
<code>network.allowMachLookup</code>	Additional XPC/Mach service names the sandbox may look up (macOS only). Supports a single trailing <code>*</code> for prefix matching. Needed for tools that communicate via XPC such as the iOS Simulator or Playwright.	<code>["com.apple.coresimulator.*"]</code>

Keys	Description	Example
<code>network.allowedDomains</code>	Array of domains to allow for outbound network traffic. Supports wildcards (e.g., <code>*.example.com</code>).	<code>["github.com", "*.npmjs.org"]</code>
<code>network.deniedDomains</code>	Array of domains to block for outbound network traffic. Supports the same wildcard syntax as <code>allowedDomains</code> . Takes precedence over <code>allowedDomains</code> when both match. Merged from all settings sources regardless of <code>allowManagedDomainsOnly</code> .	<code>["sensitive.cloud.example.com"]</code>
<code>network.allowManagedDomainsOnly</code>	(Managed settings only) Only <code>allowedDomains</code> and <code>WebFetch(domain:...)</code> allow rules from managed settings are respected. Domains from user, project, and local settings are ignored. Non-allowed domains are blocked automatically without prompting the user. Denied domains are still respected from all sources. Default: false	<code>true</code>
<code>network.httpProxyPort</code>	HTTP proxy port used if you wish to bring your own proxy. If not specified, Claude will run its own proxy.	<code>8080</code>
<code>network.socksProxyPort</code>	SOCKS5 proxy port used if you wish to bring your own proxy. If not specified, Claude will run its own proxy.	<code>8081</code>
<code>enableWeakerNestedSandbox</code>	Enable weaker sandbox for unprivileged Docker environments (Linux and WSL2 only). Reduces security . Default: false	<code>true</code>
<code>enableWeakerNetworkIsolation</code>	(macOS only) Allow access to the system TLS trust service (<code>com.apple.trustd.agent</code>) in the sandbox. Required for Go-based tools like <code>gh</code> , <code>gcloud</code> , and <code>terraform</code> to verify TLS certificates when using <code>httpProxyPort</code> with a MITM proxy and custom CA. Reduces security by opening a potential data exfiltration path. Default: false	<code>true</code>
<code>allowAppleEvents</code>	(macOS only) Allow sandboxed commands to send Apple Events. Required for <code>open</code> , <code>osascript</code> , and tools that open URLs in a browser, which otherwise fail with error <code>-600</code> . Removes code-execution isolation . Sandboxed commands can launch other applications unsandboxed with no user prompt; they can also send AppleScript commands to running applications such as Terminal, subject to the per-app macOS automation-consent prompt (TCC). Only honored from user, managed, or CLI settings, not from project settings. Default: false	<code>true</code>
<code>bwrapPath</code>	(Managed settings only, Linux/WSL2) Absolute path to the bubblewrap (<code>bwrap</code>) binary. Overrides automatic detection via <code>PATH</code> . Only honored from <u>managed settings</u> , not from user or project settings. Useful when <code>bwrap</code> is installed at a non-standard location in managed environments.	<code>/opt/admin/bwrap</code>
<code>socatPath</code>	(Managed settings only, Linux/WSL2) Absolute path to the <code>socat</code> binary used for the sandbox network proxy. Overrides automatic detection via <code>PATH</code> . Only honored from managed settings.	<code>/opt/admin/socat</code>

Sandbox path prefixes

Paths in `filesystem.allowWrite` , `filesystem.denyWrite` , `filesystem.denyRead` , `filesystem.allowRead` , and `credentials.files` support these prefixes:

Prefix	Meaning	Example
<code>/</code>	Absolute path from filesystem root	<code>/tmp/build</code> stays <code>/tmp/build</code>
<code>~/</code>	Relative to home directory	<code>~/.kube</code> becomes <code>\$HOME/.kube</code>
<code>./</code> or no prefix	Relative to the project root for project settings, or to <code>~/.claude</code> for user settings	<code>./output</code> in <code>.claude/settings.json</code> resolves to <code><project-root>/output</code>

The older `//path` prefix for absolute paths still works. If you previously used single-slash `/path` expecting project-relative resolution, switch to `./path` .

This syntax differs from Read and Edit permission rules, which use `//path` for absolute and `/path` for project-relative. Sandbox filesystem paths use standard conventions: `/tmp/build` is an absolute path.

Configuration example:

```

{
  "sandbox": {
    "enabled": true,
    "autoAllowBashIfSandboxed": true,
    "excludedCommands": ["docker *"],
    "filesystem": {
      "allowWrite": ["/tmp/build", "~/.kube"],
      "denyRead": [ "~/.aws/credentials" ]
    },
    "network": {
      "allowedDomains": ["github.com", "*.npmjs.org",
"registry.yarnpkg.com"],
      "deniedDomains": ["uploads.github.com"],
      "allowUnixSockets": [
        "/var/run/docker.sock"
      ],
      "allowLocalBinding": true
    }
  }
}

```

Filesystem and network restrictions can be configured in two ways that are merged together:

- `sandbox.filesystem` **settings** (shown above): Control paths at the OS-level sandbox boundary. These restrictions apply to all subprocess commands (e.g., `kubectl` , `terraform` , `npm`), not just Claude's file tools.
- **Permission rules**: Use `Edit` allow/deny rules to control Claude's file tool access, `Read` deny rules to block reads, and `WebFetch` allow/deny rules to control network domains. Paths from these rules are also merged into the sandbox configuration.

Attribution settings

Claude Code adds attribution to git commits and pull requests. These are configured separately:

- Commits use **git trailers** (like `Co-Authored-By`) by default, which can be customized or disabled
- Pull request descriptions are plain text

Keys	Description
<code>commit</code>	Attribution for git commits, including any trailers. Empty string hides commit attribution
<code>pr</code>	Attribution for pull request descriptions. Empty string hides pull request attribution
<code>sessionUrl</code>	Whether to append the claude.ai session link as a <code>Claude-Session</code> trailer on commits and a link in pull request descriptions when running from a web or Remote Control session. Defaults to <code>true</code> . Set to <code>false</code> to omit the link

Default commit attribution:

```
Co-Authored-By: Claude Sonnet 4.6 <noreply@anthropic.com>
```

The model name in the trailer reflects the active model for the session.

Default pull request attribution:

```
 Generated with [Claude Code](https://claude.com/claude-code)
```

Example:

```
{
  "attribution": {
    "commit": "Generated with AI\n\nCo-Authored-By: AI\n\n<ai@example.com>",
    "pr": ""
  }
}
```

ⓘ The `attribution` setting takes precedence over the deprecated `includeCoAuthoredBy` setting. To hide all attribution, set `commit` and `pr` to empty strings and `sessionUrl` to `false`.

File suggestion settings

Configure a custom command for `@` file path autocomplete. The built-in file suggestion uses fast filesystem traversal, but large monorepos may benefit from project-specific indexing such as a pre-built file index or custom tooling.

```
{
  "fileSuggestion": {
    "type": "command",
    "command": "~/.claude/file-suggestion.sh"
  }
}
```

The command runs with the same environment variables as hooks, including `CLAUDE_PROJECT_DIR`. It receives JSON via stdin with a `query` field:

```
{"query": "src/comp"}
```

Output newline-separated file paths to stdout (currently limited to 15):

```
src/components/Button.tsx
src/components/Modal.tsx
src/components/Form.tsx
```

Example:

```
#!/bin/bash
query=$(cat | jq -r '.query')
# Replace your-repo-file-index with your own file search command
your-repo-file-index --query "$query" | head -20
```

Footer link badges

The `footerLinksRegexes` setting renders extra clickable badges in the footer below the input box. Use it to turn IDs printed by project CLIs, such as review tools and issue trackers, into session links.

Each entry's `pattern` regex is matched against turn output: tool results, including file contents and fetched pages, and Claude's own responses. `{name}` placeholders in `url` and `label` are filled from named capture groups in the pattern.

The following example renders a badge whenever an issue key like `PROJ-1234` appears in turn output. The `(?<key>...)` named group captures the key, and `{key}` substitutes it into the URL and label:

~/claude/settings.json

```
{
  "footerLinksRegexes": [
    {
      "type": "regex",
      "pattern": "\\b(?<key>PROJ-\\d+)\\b",
      "url": "https://issues.example.com/browse/{key}",
      "label": "{key}"
    }
  ]
}
```

With this configured, when PROJ-1234 appears in a tool result or in Claude’s reply, a PROJ-1234 badge appears in the footer linking to https://issues.example.com/browse/PROJ-1234 .

The following constraints apply to each entry:

Constraint	Behavior
URL origin	Captured values are URL-encoded and the constructed URL must share the template’s literal origin. A capture can fill a path segment or query value but cannot change where the link points
URL length	Constructed URLs longer than 2048 characters are dropped
URL scheme	Must be https , http , or a recognized editor or workspace deep-link scheme: vscode , vscode-insiders , cursor , windsurf , zed , jetbrains , idea , slack , linear , notion , figma
Label	Defaults to the matched text and is truncated to 28 display columns
Badge count	At most 5 badges render. The oldest is displaced by newer matches and /clear removes them
Settings scope	Read from user settings, the --settings flag, and managed settings only. Ignored in project .claude/settings.json and local .claude/settings.local.json

When a turn completes, Claude Code matches each entry’s pattern regex against the turn output on the main thread, so a slow regex blocks the UI until it finishes. Nested quantifiers such as (a+)+\$ can take exponentially long against certain inputs and freeze the session, so keep each pattern linear and avoid nesting + or * .

Footer badges render alongside a **custom status line** when one is configured; neither replaces the other. Use a status line for a script-driven row that computes its own content from session data, and

footer badges to turn IDs from the conversation into links without a script.

Hook configuration

These settings control which hooks are allowed to run and what HTTP hooks can access. The `allowManagedHooksOnly` setting can only be configured in managed settings. The URL and env var allowlists can be set at any settings level and merge across sources.

Behavior when `allowManagedHooksOnly` is `true` :

- Managed hooks and SDK hooks are loaded
- Hooks from plugins force-enabled in managed settings `enabledPlugins` are loaded. This lets administrators distribute vetted hooks through an organization marketplace while blocking everything else. Trust is granted by full `plugin@marketplace` ID, so a plugin with the same name from a different marketplace stays blocked
- User hooks, project hooks, and all other plugin hooks are blocked

Restrict HTTP hook URLs:

Limit which URLs HTTP hooks can target. Supports `*` as a wildcard for matching. When the array is defined, HTTP hooks targeting non-matching URLs are silently blocked. Hostname matching is case-insensitive and ignores a trailing FQDN dot, matching DNS semantics.

```
{
  "allowedHttpHookUrls": ["https://hooks.example.com/*",
    "http://localhost:*"]
}
```

Restrict HTTP hook environment variables:

Limit which environment variable names HTTP hooks can interpolate into header values. Each hook's effective `allowedEnvVars` is the intersection of its own list and this setting.

```
{
  "httpHookAllowedEnvVars": ["MY_TOKEN", "HOOK_SECRET"]
}
```

Compute managed settings with a policy helper

The `policyHelper` setting points at an executable that computes managed settings at startup, so admins can derive policy from device posture, identity, or a remote service instead of a static file. Configure it from MDM or a system `managed-settings.json` file. Claude Code ignores `policyHelper` when it appears in any other scope, including user settings, project settings, the HKCU registry hive, and server-managed settings.

The setting accepts these keys:

Key	Type	Description
<code>path</code>	string	Absolute path to the helper executable
<code>timeoutMs</code>	number	How long to wait for the helper before treating the run as failed
<code>refreshIntervalMs</code>	number	How often to re-run the helper in the background. Set to <code>0</code> to disable refresh, or to at least <code>60000</code>

The helper writes a JSON envelope to stdout. Put the settings under a `managedSettings` key rather than at the top level, since a bare settings object parses with `managedSettings` undefined and applies nothing:

```
{
  "managedSettings": {
    "permissions": { "deny": ["Read(//etc/secrets/**)"] }
  },
  "claudeMd": "# Organization context\n...",
  "appendSystemPrompt": "Always cite the internal style guide."
}
```

When the helper emits `managedSettings`, that object replaces the file-based managed settings for the run. When the helper exits non-zero at startup, Claude Code prints the error and refuses to start, so a helper that needs outage resilience should serve from its own cache and exit `0`.

Settings precedence

Settings apply in order of precedence. From highest to lowest:

1. **Managed settings** (server-managed, MDM/OS-level policies, or managed settings)
 - Policies deployed by IT through server delivery, MDM configuration profiles, registry

policies, or managed settings files

- Cannot be overridden by any other level, including command line arguments
- Within the managed tier, precedence is: server-managed > MDM/OS-level policies > file-based (`managed-settings.d/*.json` + `managed-settings.json`) > HKCU registry (Windows only). Only one managed source is used; sources do not merge across tiers. Within the file-based tier, drop-in files and the base file are merged together.
- Embedding hosts such as Claude Desktop can supply policy via the SDK `managedSettings` option. By default this is ignored when an admin-deployed managed source is present: server-managed settings, an MDM or OS-level policy, or a managed settings file. The user-writable HKCU registry fallback does not count as an admin-deployed source. Administrators can opt in by setting `parentSettingsBehavior` to `"merge"`. The embedder's values are filtered so they can tighten managed policy but not loosen it.

2. Command line arguments

- Temporary overrides for a specific session. JSON passed via `--settings <file-or-json>` merges with file-based settings using the same rules as the other layers: a key set here overrides the same key in local, project, or user settings, and omitting a key leaves the lower-layer value in place

3. Local project settings (`.claude/settings.local.json`)

- Personal project-specific settings

4. Shared project settings (`.claude/settings.json`)

- Team-shared project settings in source control

5. User settings (`~/.claude/settings.json`)

- Personal global settings

This hierarchy ensures that organizational policies are always enforced while still allowing teams and individuals to customize their experience. The same precedence applies whether you run Claude Code from the CLI, the VS Code extension, or a JetBrains IDE.

For example, if your user settings set `permissions.defaultMode` to `acceptEdits` and a project's shared settings set it to `default`, the project value applies. The example below covers how array-valued settings such as permission rules combine instead.

❗ **Array settings merge across scopes.** When the same array-valued setting (such as `sandbox.filesystem.allowWrite` or `permissions.allow`) appears in multiple scopes, the arrays are **concatenated and deduplicated**, not replaced. This means lower-priority scopes can add entries without overriding those set by higher-priority scopes, and vice versa. For example, if managed settings set `allowWrite` to `["/opt/company-tools"]` and a user adds `["~/ .kube"]`, both paths are included in the final configuration.

Two array settings do not merge this way:

- `fallbackModel` is an ordered chain where position carries meaning: the highest-precedence file that defines it supplies the entire value.
- `availableModels`: when the **highest-precedence managed source** defines it, that list applies as-is and user, project, and local entries cannot extend it. Across non-managed scopes the arrays merge as usual. See [Merge behavior](#).

Verify active settings

Run `/status` inside Claude Code to see which settings sources are active. Inside the menu, the **Status** tab includes a `Setting sources` line that lists each layer Claude Code loaded for the current session, such as `User settings` or `Project local settings`. When **managed settings** are in effect, the entry shows the delivery channel in parentheses, for example `Enterprise managed settings (remote)`, `(plist)`, `(HKLM)`, `(HKCU)`, or `(file)`. A layer appears in the list only when that source is loaded with at least one key, so an empty list means no settings sources were found.

The `Setting sources` line confirms which sources are being read. It does not show which layer supplied each individual key. The **Config** tab in the same dialog is an editor for a fixed set of toggles such as theme and verbose output, not a view of your `settings.json` contents.

If a settings file contains errors, such as invalid JSON or a value that fails validation, `/status` lists the affected files. Run `/doctor` to see the details for each error.

Key points about the configuration system

- **Memory files** (`CLAUDE.md`): Contain instructions and context that Claude loads at startup
- **Settings files (JSON)**: Configure permissions, environment variables, and tool behavior
- **Skills**: Custom prompts that can be invoked with `/skill-name` or loaded by Claude automatically
- **MCP servers**: Extend Claude Code with additional tools and integrations
- **Precedence**: Higher-level configurations (Managed) override lower-level ones (User/Project)

- **Inheritance:** Settings merge across scopes; scalar values from higher-priority scopes override, and arrays concatenate, with two exceptions described in the [array-merge Note](#)

System prompt

Claude Code's internal system prompt is not published. To add custom instructions, use `CLAUDE.md` files or the `--append-system-prompt` flag.

Exclude sensitive files

To prevent Claude Code from accessing files containing sensitive information like API keys, secrets, and environment files, use the `permissions.deny` setting in your `.claude/settings.json` file:

```
{
  "permissions": {
    "deny": [
      "Read(./env)",
      "Read(./env.*)",
      "Read(./secrets/**)",
      "Read(./config/credentials.json)",
      "Read(./build)"
    ]
  }
}
```

This replaces the deprecated `ignorePatterns` configuration. Files matching these patterns are excluded from file discovery and search results, and read operations on these files are denied.

Subagent configuration

Claude Code supports custom AI subagents that can be configured at both user and project levels. These subagents are stored as Markdown files with YAML frontmatter:

- **User subagents:** `~/claude/agents/` , available across all your projects
- **Project subagents:** `.claude/agents/` , specific to your project and shareable with your team

Subagent files define specialized AI assistants with custom prompts and tool permissions. Learn more about creating and using subagents in the [subagents documentation](#).

Plugin configuration

Claude Code supports a plugin system that lets you extend functionality with skills, agents, hooks, and MCP servers. Plugins are distributed through marketplaces and can be configured at both user and repository levels.

Plugin settings

Plugin-related settings in `settings.json` :

```
{
  "enabledPlugins": {
    "formatter@acme-tools": true,
    "deployer@acme-tools": true,
    "analyzer@security-plugins": false
  },
  "extraKnownMarketplaces": {
    "acme-tools": {
      "source": {
        "source": "github",
        "repo": "acme-corp/claude-plugins"
      }
    }
  }
}
```

`enabledPlugins`

Controls which plugins are enabled. Format: `"plugin-name@marketplace-name": true/false` . A plugin with no entry at any scope falls back to its `defaultEnabled` value.

Scopes:

- **User settings** (`~/.claude/settings.json`): Personal plugin preferences
- **Project settings** (`.claude/settings.json`): Project-specific plugins shared with team
- **Local settings** (`.claude/settings.local.json`): Per-machine overrides, gitignored when Claude Code creates it
- **Managed settings** (`managed-settings.json`): Organization-wide policy overrides that block installation at all scopes and hide the plugin from the marketplace

❗ Project settings take precedence over user settings, so setting a plugin to `false` in `~/.claude/settings.json` does not disable a plugin that the project's `.claude/settings.json` enables. To opt out of a project-enabled plugin on your machine, set it to `false` in `.claude/settings.local.json` instead.

Plugins force-enabled by managed settings cannot be disabled this way, since managed settings override local settings.

Example:

```
{
  "enabledPlugins": {
    "code-formatter@team-tools": true,
    "deployment-tools@team-tools": true,
    "experimental-features@personal": false
  }
}
```

`extraKnownMarketplaces`

Defines additional marketplaces that should be made available for the repository. Typically used in repository-level settings to ensure team members have access to required plugin sources.

When a repository includes `extraKnownMarketplaces` :

1. Team members are prompted to install the marketplace when they trust the folder
2. Team members are then prompted to install plugins from that marketplace
3. Users can skip unwanted marketplaces or plugins (stored in user settings)
4. Installation respects trust boundaries and requires explicit consent

Example:

```

{
  "extraKnownMarketplaces": {
    "acme-tools": {
      "source": {
        "source": "github",
        "repo": "acme-corp/claude-plugins"
      }
    },
    "security-plugins": {
      "source": {
        "source": "git",
        "url": "https://git.example.com/security/plugins.git"
      }
    }
  }
}

```

Marketplace source types:

- `github` : GitHub repository (uses `repo`)
- `git` : Any git URL (uses `url`)
- `directory` : Local filesystem path (uses `path` , for development only)
- `hostPattern` : regex pattern to match marketplace hosts (uses `hostPattern`)
- `settings` : inline marketplace declared directly in settings.json without a separate hosted repository (uses `name` and `plugins`)

The `git` source type works with any git hosting service, including self-hosted GitLab and Bitbucket. Claude Code clones the repository with the same authentication that `git clone` would use on that machine: configured credential helpers, SSH keys, or a host-specific token environment variable. See [Private repositories](#) for setup details.

For `github` and `git` sources, set `"skipLfs": true` inside the `source` object (alongside `repo` or `url`) to skip Git LFS downloads when Claude Code clones or updates the marketplace repository. LFS pointer files remain as pointers instead of downloading their content. Use this when the repository contains large LFS objects unrelated to plugin content. Requires Claude Code v2.1.153 or later.

Each marketplace entry also accepts an optional `autoUpdate` Boolean. Set `"autoUpdate": true` alongside `source` to make Claude Code refresh that marketplace and update its installed plugins at

startup. When omitted, official Anthropic marketplaces default to `true` and all other marketplaces default to `false`. See [Configure auto-updates](#).

Use `source: 'settings'` to declare a small set of plugins inline without setting up a hosted marketplace repository. Plugins listed here must reference external sources such as GitHub or npm. You still need to enable each plugin separately in `enabledPlugins`.

```
{
  "extraKnownMarketplaces": {
    "team-tools": {
      "source": {
        "source": "settings",
        "name": "team-tools",
        "plugins": [
          {
            "name": "code-formatter",
            "source": {
              "source": "github",
              "repo": "acme-corp/code-formatter"
            }
          }
        ]
      }
    }
  }
}
```

`strictKnownMarketplaces`

Managed settings only: Controls which plugin marketplaces users are allowed to add and install plugins from. This setting can only be configured in [managed settings](#) and provides administrators with strict control over marketplace sources.

Managed settings file locations:

- **macOS:** `/Library/Application Support/ClaudeCode/managed-settings.json`
- **Linux and WSL:** `/etc/claude-code/managed-settings.json`
- **Windows:** `C:\Program Files\ClaudeCode\managed-settings.json`

Key characteristics:

- Only available in managed settings (`managed-settings.json`)

- Cannot be overridden by user or project settings (highest precedence)
- Enforced before network and filesystem operations, so blocked sources never run
- Uses exact matching for source specifications (including `ref` , `path` for git sources), except `hostPattern` and `pathPattern` , which use regex matching

Allowlist behavior:

- `undefined` (default): no restrictions, so users can add any marketplace
- Empty array `[]` : complete lockdown, so users can't add any new marketplaces
- List of sources: users can only add marketplaces that match exactly

All supported source types:

The allowlist supports multiple marketplace source types. Most sources use exact matching, while `hostPattern` and `pathPattern` use regex matching against the marketplace host and filesystem path respectively.

1. GitHub repositories:

```
{ "source": "github", "repo": "acme-corp/approved-plugins" }
{ "source": "github", "repo": "acme-corp/security-tools", "ref": "v2.0" }
{ "source": "github", "repo": "acme-corp/plugins", "ref": "main", "path": "marketplace" }
```

Fields: `repo` (required), `ref` (optional: branch/tag/SHA), `path` (optional: subdirectory)

2. Git repositories:

```
{ "source": "git", "url": "https://gitlab.example.com/tools/plugins.git" }
{ "source": "git", "url": "https://bitbucket.org/acme-corp/plugins.git", "ref": "production" }
{ "source": "git", "url": "ssh://git@git.example.com/plugins.git", "ref": "v3.1", "path": "approved" }
```

Fields: `url` (required), `ref` (optional: branch/tag/SHA), `path` (optional: subdirectory)

3. URL-based marketplaces:

```
{ "source": "url", "url":  
  "https://plugins.example.com/marketplace.json" }  
{ "source": "url", "url":  
  "https://cdn.example.com/marketplace.json", "headers": {  
    "Authorization": "Bearer ${TOKEN}" } }
```

Fields: `url` (required), `headers` (optional: HTTP headers for authenticated access)

❗ URL-based marketplaces only download the `marketplace.json` file. They do not download plugin files from the server. Plugins in URL-based marketplaces must use external sources (GitHub, npm, or git URLs) rather than relative paths. For plugins with relative paths, use a Git-based marketplace instead. See [Troubleshooting](#) for details.

4. NPM packages:

```
{ "source": "npm", "package": "@acme-corp/claude-plugins" }  
{ "source": "npm", "package": "@acme-corp/approved-marketplace" }
```

Fields: `package` (required, supports scoped packages)

5. File paths:

```
{ "source": "file", "path": "/usr/local/share/claude/acme-  
marketplace.json" }  
{ "source": "file", "path": "/opt/acme-  
corp/plugins/marketplace.json" }
```

Fields: `path` (required: absolute path to marketplace.json file)

6. Directory paths:

```
{ "source": "directory", "path": "/usr/local/share/claude/acme-  
plugins" }  
{ "source": "directory", "path": "/opt/acme-corp/approved-  
marketplaces" }
```

Fields: `path` (required: absolute path to directory containing `.claude-plugin/marketplace.json`)

7. Host pattern matching:

```
{ "source": "hostPattern", "hostPattern":  
  "^github\\.example\\.com$" }  
{ "source": "hostPattern", "hostPattern":  
  "^gitlab\\.internal\\.example\\.com$" }
```

Fields: `hostPattern` (required: regex pattern to match against the marketplace host)

Use host pattern matching when you want to allow all marketplaces from a specific host without enumerating each repository individually. This is useful for organizations with internal GitHub Enterprise or GitLab servers where developers create their own marketplaces.

Host extraction by source type:

- `github` : always matches against `github.com`
- `git` : extracts hostname from the URL (supports both HTTPS and SSH formats)
- `url` : extracts hostname from the URL
- `npm` , `file` , `directory` : not supported for host pattern matching

8. Path pattern matching:

```
{ "source": "pathPattern", "pathPattern": "^/opt/approved/" }  
{ "source": "pathPattern", "pathPattern": ".*" }
```

Fields: `pathPattern` (required: regex pattern matched against the `path` field of `file` and `directory` sources)

Use path pattern matching to allow filesystem-based marketplaces alongside `hostPattern` restrictions for network sources. Set `".*"` to allow all local paths, or a narrower pattern to restrict to specific directories.

Configuration examples:

Example: allow specific marketplaces only:

```
{
  "strictKnownMarketplaces": [
    {
      "source": "github",
      "repo": "acme-corp/approved-plugins"
    },
    {
      "source": "github",
      "repo": "acme-corp/security-tools",
      "ref": "v2.0"
    },
    {
      "source": "url",
      "url": "https://plugins.example.com/marketplace.json"
    },
    {
      "source": "npm",
      "package": "@acme-corp/compliance-plugins"
    }
  ]
}
```

Example: disable all marketplace additions:

```
{
  "strictKnownMarketplaces": []
}
```

Example: allow all marketplaces from an internal git server:

```
{
  "strictKnownMarketplaces": [
    {
      "source": "hostPattern",
      "hostPattern": "^github\\.example\\.com$"
    }
  ]
}
```

Exact matching requirements:

Marketplace sources must match exactly for a user’s addition to be allowed. For git-based sources (`github` and `git`), this includes all optional fields:

- The `repo` or `url` must match exactly
- The `ref` field must match exactly (or both be undefined)
- The `path` field must match exactly (or both be undefined)

Examples of sources that don’t match:

```
// These are DIFFERENT sources:
{ "source": "github", "repo": "acme-corp/plugins" }
{ "source": "github", "repo": "acme-corp/plugins", "ref": "main"
}

// These are also DIFFERENT:
{ "source": "github", "repo": "acme-corp/plugins", "path":
"marketplace" }
{ "source": "github", "repo": "acme-corp/plugins" }
```

Comparison with `extraKnownMarketplaces` :

Aspect	<code>strictKnownMarketplaces</code>	<code>extraKnownMarketplaces</code>
Purpose	Organizational policy enforcement	Team convenience
Settings file	<code>managed-settings.json</code> only	Any settings file
Behavior	Blocks non-allowlisted additions	Auto-installs missing marketplaces
When enforced	Before network/filesystem operations	After user trust prompt
Can be overridden	No (highest precedence)	Yes (by higher precedence settings)
Source format	Direct source object	Named marketplace with nested source
Use case	Compliance, security restrictions	Onboarding, standardization

Format difference:

`strictKnownMarketplaces` uses direct source objects:

```
{
  "strictKnownMarketplaces": [
    { "source": "github", "repo": "acme-corp/plugins" }
  ]
}
```

`extraKnownMarketplaces` requires named marketplaces:

```
{
  "extraKnownMarketplaces": {
    "acme-tools": {
      "source": { "source": "github", "repo": "acme-corp/plugins"
    }
  }
}
```

Using both together:

`strictKnownMarketplaces` is a policy gate: it controls what users may add but does not register any marketplaces. To both restrict and pre-register a marketplace for all users, set both in `managed-settings.json` :

```
{
  "strictKnownMarketplaces": [
    { "source": "github", "repo": "acme-corp/plugins" }
  ],
  "extraKnownMarketplaces": {
    "acme-tools": {
      "source": { "source": "github", "repo": "acme-corp/plugins"
    }
  }
}
```

With only `strictKnownMarketplaces` set, users can still add the allowed marketplace manually via `/plugin marketplace add` , but it is not available automatically.

Important notes:

- Restrictions are checked before any network requests or filesystem operations
- When blocked, users see clear error messages indicating the source is blocked by managed policy
- The restriction is enforced on marketplace add and on plugin install, update, refresh, and auto-update. A marketplace added before the policy was set cannot be used to install or update plugins once its source no longer matches the allowlist
- Managed settings have the highest precedence and cannot be overridden

See [Managed marketplace restrictions](#) for user-facing documentation.

`strictPluginOnlyCustomization`

Managed settings only: blocks skills, agents, hooks, and MCP servers from user and project sources, so they can only come from plugins or managed settings. Combine it with `strictKnownMarketplaces` to control the full customization supply chain: the marketplace allowlist controls which plugins users can install, and this setting blocks everything that doesn't come from a plugin or from managed settings.

⚠ `strictPluginOnlyCustomization` requires Claude Code v2.1.82 or later. Earlier versions ignore the key and keep loading user and project customizations, so the lockdown isn't enforced until clients update.

The value is either `true` to lock all four surfaces, or an array naming the surfaces to lock:

```
{
  "strictPluginOnlyCustomization": ["skills", "hooks"]
}
```

For each locked surface, Claude Code skips user-level and project-level sources and loads only plugin-provided and managed sources:

Surface	Blocked when locked	Still loads
skills	<code>~/.claude/skills/</code> , <code>.claude/skills/</code>	Plugin skills, bundled skills, skills in the managed policy directory
agents	<code>~/.claude/agents/</code> , <code>.claude/agents/</code>	Plugin agents, built-in agents, agents in the managed policy directory
hooks	Hooks in user, project, and local <code>settings.json</code>	Plugin hooks, hooks in managed settings

Surface	Blocked when locked	Still loads
mcp	Servers in <code>~/.claude.json</code> and <code>.mcp.json</code>	Plugin MCP servers, <u>managed-mcp.json</u> servers

Surface names that a Claude Code version doesn't recognize are ignored rather than failing the settings file, so you can add new surface names before all clients have updated.

Manage plugins

Use the `/plugin` command to manage plugins interactively:

- Browse available plugins from marketplaces
- Install/uninstall plugins
- Enable/disable plugins
- View plugin details (skills, agents, hooks provided)
- Add/remove marketplaces

Learn more about the plugin system in the [plugins documentation](#).

Environment variables

Environment variables let you control Claude Code behavior without editing settings files. Any variable can also be configured in settings.json under the `env` key to apply it to every session or roll it out to your team.

See the [environment variables reference](#) for the full list.

Tools available to Claude

Claude Code has access to a set of tools for reading, editing, searching, running commands, and orchestrating subagents. Tool names are the exact strings you use in permission rules and hook matchers.

See the [tools reference](#) for the full list and Bash tool behavior details.

See also

- **Permissions**: permission system, rule syntax, tool-specific patterns, and managed policies
- **Authentication**: set up user access to Claude Code
- **Debug your configuration**: diagnose why a setting, hook, or MCP server isn't taking effect
- **Troubleshoot installation and login**: installation, authentication, and platform issues